

```

/**
 * GymResult Proxy Server
 * Gira su Render/Railway - risolve il problema CORS
 * chiamando live.gr.php server-side e restituendo i dati puliti.
 */

const express = require("express");
const fetch = require("node-fetch");
const cors = require("cors");

const app = express();
const PORT = process.env.PORT || 3000;

// Abilita CORS per tutti (così l'app Claude o qualsiasi frontend può chiamarlo)
app.use(cors());
app.use(express.json());

// — Endpoint principale —————
// GET /live?gara=28831
// Chiama GymResult, parsando l'HTML, restituisce JSON pulito con le atlete.
app.get("/live", async (req, res) => {
  const { gara, categoria = "*", attrezzo = "*", turno = "*" } = req.query;

  if (!gara) {
    return res.status(400).json({ error: "Parametro 'gara' obbligatorio" });
  }

  try {
    const body = new URLSearchParams({
      sezione: "AF",
      fase: "",
      regione: "",
      gara,
      attrezzo,
      categoria,
      turno,
    }).toString();

    const response = await fetch("https://www.gymresult.it/live.gr.php", {
      method: "POST",
      headers: {
        "Content-Type": "application/x-www-form-urlencoded",
        Accept: "application/json, text/javascript, */*; q=0.01",
        "X-Requested-With": "XMLHttpRequest",
      },
    });
  }
});

```

```

    Referer: `https://www.gymresult.it/live/`,
    "User-Agent":
      "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Origin: "https://www.gymresult.it",
  },
  body,
});

if (!response.ok) {
  return res
    .status(502)
    .json({ error: `GymResult ha risposto con ${response.status}` });
}

const data = await response.json();

// Il campo "cat" contiene l'HTML della classifica di categoria
const html = data.cat || data.spe || "";
const titolo = data.int || "";

if (!html) {
  return res.json({ titolo, atlete: [], raw: data });
}

// Parsa l'HTML per estrarre le atlete
const atlete = parseGymResultHTML(html);

res.json({
  titolo,
  gara,
  timestamp: new Date().toISOString(),
  atlete,
  // Includi anche i campi grezzi per debug
  _raw_keys: Object.keys(data),
});
} catch (err) {
  console.error("Errore proxy:", err);
  res.status(500).json({ error: err.message });
}
});

// — Endpoint debug: restituisce il JSON grezzo di GymResult —————
app.get("/raw", async (req, res) => {
  const { gara = "28831" } = req.query;

  const body = new URLSearchParams({
    sezione: "AF",
  });

```

```

    fase: "",
    regione: "",
    gara,
    attrezzo: "*",
    categoria: "*",
    turno: "*",
  }).toString();

  try {
    const response = await fetch("https://www.gymresult.it/live.gr.php", {
      method: "POST",
      headers: {
        "Content-Type": "application/x-www-form-urlencoded",
        Accept: "application/json, text/javascript, */*; q=0.01",
        "X-Requested-With": "XMLHttpRequest",
        Referer: "https://www.gymresult.it/live/",
        "User-Agent":
          "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 Chrome/143.0.0.0 Sa
          Origin: "https://www.gymresult.it",
      },
      body,
    });

    const text = await response.text();

    // Prova JSON, altrimenti restituisce testo grezzo
    try {
      res.json(JSON.parse(text));
    } catch {
      res.type("text").send(text);
    }
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// — Health check —————
app.get("/", (req, res) => {
  res.json({
    status: "ok",
    service: "GymResult Proxy — Gold GAF 2026",
    endpoints: {
      "/live?gara=28831": "Classifica parsata",
      "/raw?gara=28831": "JSON grezzo da GymResult",
    },
  });
});
});

```

```

// — Parser HTML GymResult —————
// L'HTML di GymResult ha una struttura con righe tipo:
// <div class="riga">
//   <span class="lbPos">1</span>
//   <span class="lbNome">ROSSI MARTA</span>
//   <span class="lbSoc">5/141 - A.S.D. SESTRI PONENTE</span>
//   <span class="lbPun">73,725</span>
// </div>
// Usiamo regex robusti perché non abbiamo un DOM lato server.
function parseGymResultHTML(html) {
  const atlete = [];
  const seenIds = new Set();

  // Regex per estrarre blocchi di riga atleta
  // GymResult usa classi CSS tipo lbNome, lbSoc, lbPun, lbPos
  const rigaRegex =
    /class="[^"]*lbNome["]*"^[^>]*>([\s\S]*?)<\/span>[\s\S]*?class="[^"]*lbSoc["^"
    // Cerca anche la categoria/fascia corrente prima della riga
  const catRegex = /class="[^"]*titolo["]*"^[^>]*>[\s\S]*?(ALL?IEVE\s*[345]|A[345
  // Mappa posizioni categoria
  const catMatches = [];
  let cm;
  while ((cm = catRegex.exec(html)) !== null) {
    catMatches.push({ idx: cm.index, cat: normalizzaCat(cm[1]) });
  }

  let m;
  while ((m = rigaRegex.exec(html)) !== null) {
    const nome = stripHTML(m[1]).trim().toUpperCase();
    const socRaw = stripHTML(m[2]).trim();
    const punRaw = stripHTML(m[3]).trim();

    if (!nome || !socRaw || !punRaw) continue;

    // Estrai codice e nome società
    const socMatch = socRaw.match(/^(\d+\/\d+)\s*[\-\s]*(.+)$/);
    const codice = socMatch ? socMatch[1] : null;
    const soc = socMatch ? socMatch[2].trim().toUpperCase() : socRaw.toUpperCase()

    // Determina regione dal codice
    const reg = codice ? codiceToReg(codice) : null;

    // Punteggio (formato italiano: 73,725)
    const score = parseFloat(punRaw.replace(/\.\/g, "").replace(",", "."));

```

```

    if (isNaN(score)) continue;

    // Determina categoria dalla posizione nel documento
    const cat = catFromPosition(m.index, catMatches);

    const id = `${cat}_${nome}_${codice || soc}`;
    if (seenIds.has(id)) continue;
    seenIds.add(id);

    atlete.push({ cat, nome, soc, codice, reg, score });
}

// Fallback: se la regex principale non ha trovato nulla, prova pattern alterna
if (atlete.length === 0) {
    return parseGymResultHTMLFallback(html);
}

return atlete;
}

function parseGymResultHTMLFallback(html) {
    // Pattern alternativo: cerca ogni cella con punteggio vicino a un nome
    const atlete = [];
    // Estrai tutto il testo visibile e usa il parser testuale
    const text = html
        .replace(/<script[\s\S]*?<\/script>/gi, "")
        .replace(/<style[\s\S]*?<\/style>/gi, "")
        .replace(/<[^>]+>/g, " ")
        .replace(/&amp;/g, "&")
        .replace(/&lt;/g, "<")
        .replace(/&gt;/g, ">")
        .replace(/&nbsp;/g, " ")
        .replace(/\\s+/g, "\\n")
        .trim();

    // Riusa la logica del parser testuale dell'app principale
    // (semplificato - il server ritorna il testo grezzo per permettere
    // al client di fare il parsing avanzato se necessario)
    return [{ _raw_text: text.substring(0, 5000), _parse_failed: true }];
}

function stripHTML(s) {
    return s
        .replace(/<[^>]+>/g, "")
        .replace(/&amp;/g, "&")
        .replace(/&lt;/g, "<")
        .replace(/&gt;/g, ">")

```

```

        .replace(/&nbsp;/g, " ")
        .trim();
    }

function normalizzaCat(s) {
    const m = s.match(/[345]/);
    return m ? "A" + m[0] : null;
}

function catFromPosition(idx, catMatches) {
    let cat = null;
    for (const c of catMatches) {
        if (c.idx <= idx) cat = c.cat;
        else break;
    }
    return cat;
}

const FGI_REG = {
    1: "PIE", 2: "LOM", 3: "VEN", 4: "FVG", 5: "LIG",
    6: "EMI", 7: "TOS", 8: "MAR", 9: "UMB", 10: "LAZ",
    11: "CAM", 12: "PUG", 13: "CAL", 14: "ABR", 15: "BAS",
    16: "SIC", 17: "SAR", 18: "BOL", 19: "TRE", 20: "MOL",
};

function codiceToReg(codice) {
    const m = codice.match(/^(\\d+)\\//);
    return m ? FGI_REG[parseInt(m[1])] || null : null;
}

// — Start —————
app.listen(PORT, () => {
    console.log(`✅ GymResult Proxy in ascolto su porta ${PORT}`);
});

```